llm-ada-project | documentação

Arquitetura CUDA NVIDIA

para medir e otimizar a latência de tokens

Foco: `timed_generate` e `TokenTimer` em `modelos/*.py`

Roteiro

1. SM, warp e warp scheduler
2. Hierarquia de memória
3. Lançamento de kernels, CUDA graphs, `torch.compile`
4. Sincronização host-device e streamer
5. DVFS, clocks e warm-up
6. Tensor cores e quantização 4-bit
7. Checklist para o benchmark
8. Onde achar slides da NVIDIA

GPU de referência: NVIDIA T4 (Turing, Colab). Os números são do datasheet/whitepaper; as estimativas estão marcadas como tal. Nada aqui foi executado em GPU.

O que o benchmark mede hoje

```
cuda_sync()
start = time.perf_counter()
generated = model.generate(**inputs,
    min_new_tokens=160, max_new_tokens=160,
    do_sample=False, use_cache=True,
    streamer=timer)          # TokenTimer
cuda_sync()
total = time.perf_counter() - start
ttft = timer.times[0]  - start
ttlt = timer.times[-1] - start
```

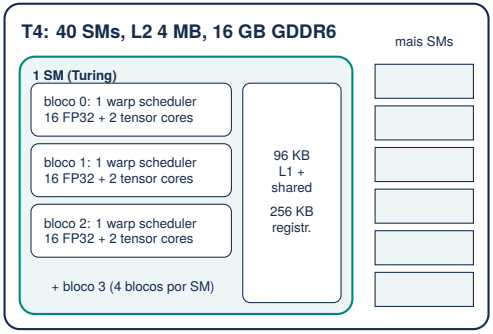
- `TokenTimer.put` anota `perf_counter()` quando cada token **chega na CPU**.
- Warm-up: 1 chamada de 8 tokens antes de cada bloco.
- `SPEED_N = 20` perguntas, batch = 1, greedy.



TTFT = prefill do prompt inteiro + 1º token.
decode tok/s = $(N-1)/(\text{TTLT}-\text{TTFT})$.

Os dois regimes usam a GPU de formas diferentes: o prefill é limitado por **compute**; o decode, por **memória e latência**. O resto do deck explica por quê.

1. Hierarquia: GPU → SM → warp → thread

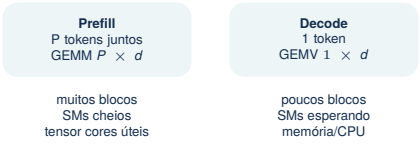


- **Kernel** = grade de **blocos**; cada bloco roda inteiro em 1 SM.
- **Warp** = 32 threads que executam a mesma instrução (SIMT).
- Cada **warp scheduler** escolhe, a cada ciclo, um warp *pronto* e despacha sua instrução.
- Um warp que espera memória ($\sim$centenas de ciclos) fica parado; o scheduler troca para outro sem custo. Isso é **latency hiding**.
- Para esconder a latência é preciso ter **muitos warps em voo** (ocupação) e trabalho independente.

Fonte: NVIDIA Turing Architecture Whitepaper; T4 datasheet.

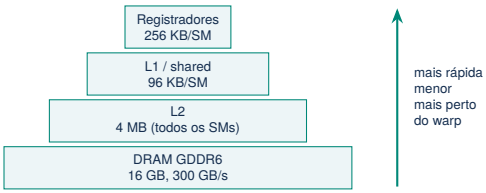
1. Por que o decode batch = 1 é *latency-bound*

- Cada passo depende do anterior: o token t só existe depois do `argmax` do passo $t-1$. Não dá para paralelizar no tempo.
- Com batch = 1, cada camada linear vira **GEMV** ($1 \times d$ por $d \times d'$): pouca matemática, muitos pesos lidos.
- Kernels pequenos (RMSNorm, RoPE, residual, softmax) lançam poucos blocos: parte dos 40 SMs fica ociosa e há poucos warps para esconder latência.
- Um passo de decode = **centenas de kernels** em sequência. O tempo por token é a soma das latências, não da vazão.



Consequência para o benchmark: o decode tok/s do projeto mede mais a *latência de memória e de lançamento* do que a capacidade de cálculo da GPU. Mudanças pequenas de overhead aparecem direto no resultado.

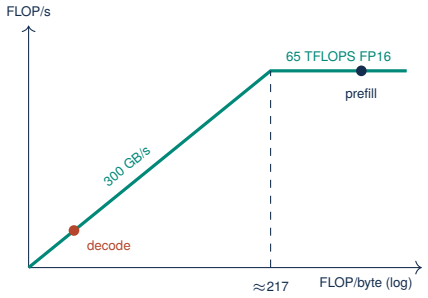
2. Hierarquia de memória (T4)



- **Registradores**: privados de cada thread; o acesso mais rápido.
- **Shared memory**: controlada pelo programador, compartilhada no bloco; é onde kernels de matmul fazem *tiling*.
- **L2**: cache comum a todos os SMs. Com 4 MB, não cabem os pesos de um modelo de 4B parâmetros: cada token lê os pesos da DRAM de novo.
- **DRAM**: a T4 usa **GDDR6**, não HBM. HBM fica nas GPUs de datacenter maiores (A100, H100); a lógica é a mesma, só a banda muda.

Turing: os 96 KB por SM dividem-se em 32/64 KB de shared + 64/32 KB de L1 (whitepaper).

2. Decode é *bandwidth-bound*, prefill é *compute-bound*



Roofline simplificado. Ponto de equilíbrio = 65e12 / 300e9 ≈ 217 FLOP/byte.

Intensidade aritmética (FLOP por byte lido):

- Decode: 2 FLOP por peso por token. Peso FP16 = 2 bytes $\Rightarrow \sim$1 FLOP/B; peso 4-bit $\Rightarrow \sim$4 FLOP/B. Muito abaixo de 217: a GPU espera a DRAM.
- Prefill com P tokens reusa cada peso P vezes $\Rightarrow \sim P\times$ mais FLOP/B. Prompts de centenas de tokens (3/5-shot) passam do equilíbrio: limite de compute.

Conta de bolso (estimativa): 4B parâmetros em 4-bit $\approx$ 2–2,5 GB lidos por token $\Rightarrow$ piso de $\sim$7–8 ms/token $\Rightarrow$ teto de $\sim$120–140 tok/s. Se o medido ficar muito abaixo, o gargalo é overhead (lançamento, Python, sync, dequant), não a banda.

O KV cache também é lido a cada token e cresce com o prompt: o 5-shot paga mais por token.

3. Overhead de lançamento de kernels



- Cada kernel custa alguns µs de CPU para ser enfileirado (Python do HF + dispatcher do PyTorch + driver).
- Por camada: norm, q/k/v, RoPE, atenção, o_proj, norm, gate/up, ativação, down, residuais (+ dequant 4-bit + LoRA no fine-tuned). Vezes dezenas de camadas.

- No decode muitos kernels duram poucos µs: a CPU não consegue enfileirar rápido o bastante e a GPU fica parada. O passo é **CPU-bound**.
- Sintoma: GPU util. alta no `nvidia-smi`, mas buracos na timeline do Nsight Systems.
- No benchmark: a condição **fine-tuned** tem adaptadores LoRA (r=8) e por isso mais kernels por token. Parte da diferença de velocidade vem daí, não do modelo.

3. CUDA graphs e `torch.compile`

- **CUDA graph**: captura a sequência de kernels de um passo uma vez e depois relança tudo com **uma** chamada. Some o custo por kernel.
- Exige formas e endereços fixos: o KV cache precisa ser **estático** (pré-alocado no tamanho máximo).
- `torch.compile(mode="reduce-overhead")` gera kernels fundidos (Triton) e usa CUDA graphs.
- Custos: compilação lenta nas primeiras chamadas e recompilação quando a forma muda. O warm-up precisa cobrir isso.

```
# KV cache fixo + forward compilado (HF)
model.generation_config.cache_implementation = "static"
model.forward = torch.compile(model.forward,
                              mode="reduce-overhead")
```

Cuidados neste projeto

- Unsloth + bitsandbytes 4-bit + LoRA com `torch.compile` não tem compatibilidade garantida: testar num piloto antes.
- Se usar, aplicar igual nos 4 modelos e nas 4 condições e registrar no manifesto; senão a comparação fica injusta.

4. Sincronização host-device

- CUDA é **assíncrono**: a CPU enfileira kernels num *stream* e segue em frente; a GPU executa depois.
- Pontos de sincronização (a CPU para e espera a GPU): `.cpu()`, `.item()`, `.tolist()`, `bool(tensor)`, `print(tensor)`, `torch.cuda.synchronize()`.
- Sem sync, `time.perf_counter()` mede quando a CPU *enfileirou*, não quando a GPU terminou. Por isso `timed_generate` chama `cuda_sync()` antes e depois.

No `generate` do Transformers (lido no código da v5.20; confira a sua versão):

- Em CUDA, cada passo já faz `unfinished_sequences.max() == 0` → `bool`: **1 sync por token**, com ou sem streamer.
- Com streamer, no mesmo ponto: `streamer.put(tokens.cpu())`, ou seja, cópia D2H + chamada Python.
- A versão que adia esse sync em 1 passo (`DeferredStopCheck`) só está ativa em MPS.

Enquanto a CPU espera o sync, ela não prepara o próximo passo. A GPU também fica parada nesse intervalo.

4. O custo do `TokenTimer` por token

```
def put(self, value):
    ids = value.tolist()            # D2H ja feito
    ...
    if self.is_prompt:
        if ids != self.prompt_ids:  # compara o prompt
            raise ValueError(...)
        ...
    self.ids.extend(ids)
    self.times.append(time.perf_counter())
```

- Por token: `.cpu()` + `tolist()` + checagens + `append`. São µs a dezenas de µs, somados aos ∼ms do passo e vezes 160 tokens.
- **1º callback = prompt inteiro**: `input_ids.cpu()`, `tolist()` e comparação de lista acontecem **dentro** da janela do TTFT. No 5-shot o prompt tem milhares de tokens, então o viés cresce com a condição.
- O carimbo é o **relógio da CPU** quando o token chega no host. Inclui ruído do Python e do GIL; não é tempo de GPU.

A checagem de consistência é boa para correção. Na medição de velocidade, dá para movê-la para depois do `generate`.

5. DVFS, clocks e warm-up

- **DVFS**: o driver muda o clock do SM e da memória conforme carga, temperatura e potência. Em repouso a GPU fica num estado de baixo clock (P8) e leva um tempo para subir.
- A T4 tem TDP de **70 W** e refrigeração passiva: em rodadas longas ela reduz o clock por potência ou temperatura. A medição varia ao longo da campanha.
- **Primeiras chamadas são mais lentas**: carga lazy de módulos CUDA, heurísticas do cuBLAS, crescimento do caching allocator, kernels do bitsandbytes e, com `torch.compile`, compilação.

Hoje no projeto

- Warm-up = **1 chamada de 8 tokens**. Ela é curta demais para estabilizar o clock e não exercita o tamanho de KV cache de 160 tokens.
- A velocidade é medida logo depois da avaliação MedQuAD, então a GPU já chega quente. A 1ª condição de cada processo, não.

Recomendado

- 3 a 5 chamadas completas (160 tokens, prompt típico da condição) antes de medir; descartar.
- Reportar mediana e IC, e checar se as primeiras perguntas são sistematicamente mais lentas.

5. Clocks estáveis com `nvidia-smi`

```
# ver clocks suportados e estado atual
nvidia-smi -q -d SUPPORTED_CLOCKS
nvidia-smi -q -d CLOCK,PERFORMANCE,POWER

# (Linux, root) manter driver carregado
sudo nvidia-smi -pm 1
# travar clock do SM num valor sustentavel
sudo nvidia-smi -lgc <MHz>,<MHz>
# ... rodar benchmark ...
sudo nvidia-smi -rgc        # destrava

# registrar clocks durante a campanha
nvidia-smi --query-gpu=timestamp,clocks.sm,\
clocks.mem,temperature.gpu,power.draw,\
clocks_event_reasons.active \
--format=csv -lms 200 > clocks.csv
```

- `-lgc`: trava o clock gráfico/SM. Use um valor da lista `SUPPORTED_CLOCKS` **abaixo do boost máximo**, que a T4 aguente em 70 W sem throttle. O resultado fica mais lento, mas repetível.
- Requer root/admin e suporte do driver. Em VMs e no Colab pode ser negado. Nesse caso, **registre** os clocks e descarte trechos com throttle.
- Em drivers antigos o campo se chama `clocks_throttle_reasons.active`.
- Salvar o `clocks.csv` junto do `manifesto.json` de cada execução.

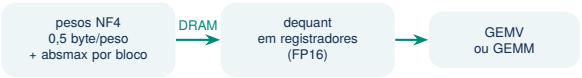
6. Tensor cores na T4

T4 (datasheet)	pico
FP32 (CUDA cores)	8,1 TFLOPS
FP16 misto (tensor cores)	65 TFLOPS
INT8 (tensor cores)	130 TOPS
INT4 (tensor cores)	260 TOPS
Banda DRAM	300 GB/s

320 tensor cores (8 por SM). Turing não tem BF16 nativo: Unsloth cai para FP16 (`is_bfloat16_supported()` = False).

- Tensor core = unidade que faz um bloco de multiplicação de matrizes (MMA) por instrução de warp. Dá $\sim 8\times$ o FP32.
- **Prefill (GEMM)**: aproveita bem os tensor cores, e o TTFT escala com o tamanho do prompt.
- **Decode (GEMV, M=1)**: o tile do tensor core fica quase vazio e o limite continua sendo a banda. Mais TFLOPS não ajudam, mas **menos bytes** ajudam.

6. Quantização 4-bit (NF4) e kernels de dequant



- `LOAD_IN_4BIT = True`: bitsandbytes NF4 (QLoRA). Cada peso ocupa 4 bits, com escala por bloco.
- **Decode**: kernel 4-bit lê 4× menos bytes que FP16 e dequantiza no próprio kernel. Ganha banda, mas paga ALU de dequant, que vale a pena quando o gargalo é memória.
- **Prefill**: em geral dequantiza a matriz para FP16 e chama a GEMM. É trabalho extra; o TTFT 4-bit pode ficar pior que o FP16.

O que isso muda na leitura dos resultados

- O decode tok/s mede *kernel 4-bit do bitsandbytes + Unsloth*, não o modelo em si. Outra versão do bnb pode mudar o número.
- Os adaptadores LoRA ficam em FP16 por cima do 4-bit: são matmuls extras por camada só na condição fine-tuned.
- Alternativas para estudar (fora do protocolo atual): kernels 4-bit dedicados para decode (ex.: Marlin/AWQ/GPTQ), ou fundir o LoRA no peso antes de medir.

O caminho exato (gemv vs dequant+GEMM) depende da versão do bitsandbytes. Registre a versão no ambiente.

7. Checklist para o benchmark

Medição (sem mudar o modelo)

- ☐ **CUDA events** para o tempo de GPU (próximo slide).
- ☐ **Warm-up maior**: 3 a 5 gerações completas de 160 tokens por condição.
- ☐ **Medir sem streamer**: tempo total com `generate` puro; TTFT numa chamada separada com `max_new_tokens=1`.
- ☐ Tirar a comparação do prompt de dentro da janela do TTFT.
- ☐ Registrar clocks, temperatura, potência e motivos de throttle (`nvidia-smi -lms`).
- ☐ Travar clock (`-lgc`) quando possível; `-pm 1` no Linux.
- ☐ Reportar mediana + IC; manter TTFT e decode separados.

Otimizações (aplicar igual em tudo)

- ☐ KV cache estático + `torch.compile(mode="reduce-overhead")` (CUDA graphs), depois de piloto.
- ☐ Atenção SDPA/FlashAttention quando o modelo suportar.
- ☐ Avaliar fundir o LoRA antes do benchmark de velocidade, ou documentar o custo extra.
- ☐ Fechar outros processos/kernels que usam a GPU.
- ☐ Perfilar 1 passo com Nsight Systems para ver os buracos entre kernels:
  `nsys profile -t cuda,nvtx python modelos/qwen35.py`

Qualquer mudança vale para os 4 modelos e as 4 condições e entra no manifesto.

7. Tempo de GPU com CUDA events (sem streamer)

```
from transformers import LogitsProcessor, LogitsProcessorList

class EventTimer(LogitsProcessor):  # 1 evento CUDA por passo, sem sync
    def __init__(self):
        self.events = []
    def __call__(self, input_ids, scores):
        ev = torch.cuda.Event(enable_timing=True)
        ev.record()
        self.events.append(ev)
        return scores
@torch.inference_mode()
def gpu_timed_generate(model, question, max_new_tokens):
    _, prompt = build_prompt(question)
    inputs = encode_prompt(prompt, max_new_tokens)
    timer = EventTimer()
    start = torch.cuda.Event(enable_timing=True)
    end = torch.cuda.Event(enable_timing=True)
    cuda_sync()
    start.record()
    model.generate(**inputs, min_new_tokens=max_new_tokens,
        max_new_tokens=max_new_tokens, do_sample=False, use_cache=True,
        pad_token_id=text_tokenizer.pad_token_id,
        logits_processor=LogitsProcessorList([timer]))
    end.record()
    cuda_sync()
    ms = [start.elapsed_time(ev) for ev in timer.events]
    return {"ttft_gpu_s": ms[0] / 1000,
            "total_gpu_s": start.elapsed_time(end) / 1000,
            "decode_tok_s": (len(ms) - 1) / ((ms[-1] - ms[0]) / 1000)}
```

- O `LogitsProcessor` roda uma vez por passo, logo depois do forward. `ev.record()` só enfileira um marcador no stream, sem sync extra.
- `elapsed_time` usa o relógio da GPU (ms). `ms[0]` $\approx$ fim do prefill = TTFT de GPU.
- Os buracos causados pela CPU continuam contando. Isso é correto: é latência real. O que sai é o ruído do Python do streamer.
- Rodar **depois** do warm-up:

```
q = speed_questions[0]
for _ in range(3):  # warm-up, descartado
    gpu_timed_generate(model, q, 160)
```

Não executado aqui (sem CUDA). Validar no Colab T4 comparando com o `TokenTimer`.

8. Onde achar slides da NVIDIA sobre arquitetura CUDA

NVIDIA On-Demand (GTC), série do Stephen Jones

- How GPU Computing Works (GTC 2021, S31151)
- How CUDA Programming Works (GTC 2022, A41101); PDF dos slides (CERN)
- How to Write a CUDA Program (GTC 2023, S51210); PDF dos slides
- The Ninja Edition (GTC 2024, S62401)
- The Parallel Programming Edition (GTC 2025, S72897)
- Busca geral: nvidia.com/on-demand. Muitas sessões têm vídeo e PDF.

Arquitetura (documentos oficiais)

- Turing Architecture Whitepaper e T4 datasheet
- Turing Tuning Guide

Cursos com slides

- OLCF/NERSC CUDA Training Series: slides e gravações apresentados por engenheiros da NVIDIA; exercícios em github.com/olcf/cuda-training-series
- NVIDIA DLI Teaching Kits: slides para professores (cadastro)
- NVIDIA DLI: cursos práticos
- NVIDIA/accelerated-computing-hub: tutoriais oficiais
- CMU 15-779, aula 3–4: GPU Architecture & CUDA Programming (Zhihao Jia, 2025)

Coletâneas

- tylera-nvidia/gpuStarterResources: índice da série GTC + slides "GPU Core Principles"
- MARD1NO/CUDA-PPT: espelho não oficial de PDFs do GTC
- gpu-mode/lectures: aulas da comunidade GPU MODE

Referências técnicas

- NVIDIA. CUDA Programming Guide; CUDA C++ Best Practices Guide.
- NVIDIA Blog. Getting Started with CUDA Graphs; How to Implement Performance Metrics in CUDA C/C++ (CUDA events).
- NVIDIA Blog. Mastering LLM Techniques: Inference Optimization (prefill vs decode, banda de memória).
- NVIDIA. nvidia-smi documentation; Nsight Systems User Guide.
- PyTorch. Accelerating PyTorch with CUDA Graphs; CUDA semantics (execução assíncrona); torch.cuda.Event.
- Hugging Face. LLM inference optimization (static cache + `torch.compile`); bitsandbytes Linear4bit.
- Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs (NF4), 2023.
- Jia et al. Dissecting the NVIDIA Turing T4 GPU via Microbenchmarking, 2019.

Código do projeto: `modelos/{gemma4,huatuogpt,medgemma,qwen35}.py`, onde `timed_generate` e `TokenTimer` são idênticos nos quatro arquivos.